Tools — Teams

Multi-agent coordination inside an active team: named teammates, a shared task list, mailbox messaging, and plan approval. All team tools require an active team, and `team_name` is required by nearly every tool.

Tool	Description
`team_create`	Create a named team from a blueprint.
`team_spawn`	Spawn a teammate for a scoped task.
`team_message`	Direct message a teammate or lead.
`team_broadcast`	Message all active teammates.
`team_read_messages`	Check mailbox for unread messages.
`team_status`	Team and task status summary: roster with each member's last five status transitions, undelivered mailbox messages, plans awaiting a verdict, and hook failures.
`team_task_list`	List all team tasks.
`team_task_create`	Lead adds a shared task.
`team_task_claim`	Teammate claims the next available task.
`team_task_complete`	Mark a claimed task done (owner or lead only).
`team_task_cancel`	Lead withdraws a `pending`/`in_progress` task.
`team_assign_task`	Lead assigns a task to a teammate.
`team_submit_plan`	Teammate submits a plan to the lead.
`team_approve_plan`	Lead approves/rejects a plan.
`team_wait`	Block until teammates finish.
`team_idle`	Teammate signals no more work.
`team_shutdown_teammate`	Lead requests teammate shutdown.
`team_shutdown_ack`	Teammate acks shutdown and exits.
`team_cleanup`	Lead tears down the team (refused while a member is live or shutting down, unless `force`).
`team_memory_read`	Read team memory bucket.
`team_memory_write`	Write team memory bucket.

Visibility switch: `teams`. See `docs/howtos/teams.md` for the full team manual and guide.

---

team_create

Create a team from a blueprint. Always pass `context` — it is prepended to every teammate's spawn prompt with the specific request details.

Arguments

Argument	Type	Required	Description	Typical value
`blueprint`	string	yes	Team template	`"code-review"`

Argument	Type	Required	Description	Typical value
`context`	string	yes	Task details: target files, what to do, where to write output	`"Review crates/ragent-server for security issues. Write findings to docs/COMPLIANCE.md"`

Workflow: `team_create` -> `team_wait` (blocks until all teammates idle) -> `team_status` to collect findings.

---

team_spawn

Add a teammate for one scoped task.

Arguments

Argument	Type	Required	Description	Typical value
`team_name`	string	yes	Team name	`"audit-team"`
`teammate_name`	string	yes	Unique teammate name	`"reviewer-1"`
`agent_type`	string	yes	Agent profile to run	`"coder"`
`prompt`	string	yes	Scoped task prompt	—

---

team_message

Argument	Type	Required	Description
`team_name`	string	yes	Team name
`to`	string	yes	Recipient teammate name (or lead)
`content`	string	yes	Message text

team_broadcast

Argument	Type	Required	Description
`team_name`	string	yes	Team name
`content`	string	yes	Message sent to all active teammates

team_read_messages

Argument	Type	Required	Description
`team_name`	string	yes	Team name

---

team_status

Team and task status summary.

Arguments

Argument	Type	Required	Description
`team_name`	string	yes	Team name

The report carries more than a roster, because a lead's question is usually "why did this teammate stop?" rather than "which members exist":

- **Per-member status history.** Each member row is followed by a `Status history` block holding that member's last five transitions with timestamps and causes, read from the team's append-only `status-history.jsonl`. The event stream carries the same information while someone is listening; the file is what makes the answer survive until the question is asked.
- **`Undelivered messages`.** A message still unread in a *terminated* teammate's mailbox is stranded - no teammate will ever drain it - so it is reported rather than left invisible.
- **`Plans awaiting a verdict`.** A plan parked in `plan_pending`, with the `team_approve_plan` call that resolves it.
- **`Hook failures`.** Configured quality-gate hooks that could not run (a refused command, a spawn failure, or the 30-second timeout), counted from `hook-failures.jsonl`. A non-zero count means a gate is not working; an inert hook never approved anything.

The same facts appear in the result metadata (`pending_plans`, `hook_failure_count`), so a programmatic reader need not parse prose.

---

team_task_list / team_task_create / team_task_claim / team_task_complete / team_assign_task

Argument	Type	Required	Applies to	Description
`team_name`	string	yes	all	Team name
`title`	string	yes	`team_task_create`	Shared task title
`task_id`	string	yes	`team_task_complete`, `team_assign_task`	Task identifier
`to`	string	yes	`team_assign_task`	Teammate to assign to

`team_task_claim` takes no extra arguments: the teammate claims the next available task from the shared list. `team_task_complete` takes `team_name` + `task_id` (not `summary`).

`team_task_complete` requires ownership. A completion is accepted only from the task's assigned agent or the lead. A task assigned to another teammate is refused and is **not** re-assigned; the refusal names the current owner and the two ways forward (the lead assigns it with `team_assign_task`, or you claim it with `team_task_claim`). An unclaimed (`pending`) task is refused the same way, and re-completing an already-completed task is a no-op success for its owner or the lead, refused for anyone else. Every refusal reports `rejection_reason` (`not_owner`, `not_claimed`, `already_completed`) with the task's `owner`, and leaves the task list untouched.

`team_task_cancel` is lead-only. It takes `team_name`, `task_id` and an optional `reason`, and is `team:manage` (the same family as assignment and cleanup). Only `pending` and `in_progress` tasks are cancellable - `completed` and `cancelled` are terminal, and the refusal names the status the task actually holds. The owner of an `in_progress` task is notified through its mailbox, and the task stays in the list with status `cancelled` rather than being deleted, so the team records that the work was dropped rather than done.

Dependencies are validated at write time. Every write to `tasks.json` validates the whole list first: a `depends_on` id naming no task in the list, or an edge that would close a dependency cycle, is refused with the offending task and chain named (`refusing to write invalid <path>: ...`) and the file left byte-identical. A dangling dependency is not repaired and not silently rewritten - fix the edge and the write proceeds.

---

team_submit_plan / team_approve_plan

Argument	Type	Required	Applies to	Description
`team_name`	string	yes	both	Team name
`plan`	string	yes	`team_submit_plan`	Plan text proposed by a teammate
`teammate`	string	yes	`team_approve_plan`	Teammate whose plan is judged
`approved`	boolean	yes	`team_approve_plan`	Approve or reject

---

team_wait / team_idle / team_shutdown_teammate / team_shutdown_ack / team_cleanup

Argument	Type	Required	Applies to	Description
`team_name`	string	yes (optional for `team_wait`, uses active team)	all	Team name
`teammate`	string	yes	`team_shutdown_teammate`	Teammate to shut down

`team_wait` blocks until spawned teammates finish — always use it after `team_create`/`team_spawn`. `team_cleanup` tears down team state after all teammates have stopped.

---

team_memory_read / team_memory_write

Argument	Type	Required	Applies to	Description
`team_name`	string	yes	both	Team name
`content`	string	yes	`team_memory_write`	Content stored in the shared memory bucket